

YEREL · GİZLİLİK-KORUMALI · YAPAY ZEKÂ

Şirket-Sorgu

Türkçe Soru → SQL → Otomatik Rapor

Şirketinizin verisine Türkçe soru sorun — kurulumunu biz yapalım.

Veriyi **hiç dışarı çıkarmadan**, tamamen yerel çalışan bir yapay zekâ ajanı: çalışan Türkçe sorar, sistem güvenli SQL üretir, çalıştırır ve doğrulanmış bir Türkçe rapor döndürür.

Hazırlayan: Harun Gökçe

Tarih: 29 Haziran 2026

Sürüm: Çalışan MVP

Bu sistemi şirketinizde kuruyoruz

Kendi veritabanınıza entegre eder, ihtiyacınıza göre özelleştirir ve eğitiriz.

GitHub (açık kaynak): github.com/Harungokc/local-text-to-sql

İletişim: harungokce70@gmail.com · 0506 155 46 42

İçindekiler

İçindekiler	2
1. Bu Sistemi Şirketinizde Kuruyoruz	3
2. Yönetici Özeti	4
3. Sorabileceğiniz Soru Türleri	5
4. Problem: Text-to-SQL Neden Zor, Gizlilik Neden Önemli?	6
5. Bu Sistemi Ne Ayırıştırıyor?	7
6. Mimari: Uçtan Uca Akış	8
7. Kontrol Katmanları: Sistemin Kalbi	10
8. Halüsinasyonsuz Rapor	11
9. Veritabanı: Neden PostgreSQL?	12
10. Model & Sunucu: Hangisi Ne Kadar Yeterli?	13
11. Yerel (Local) Çalışmanın Avantajları	14
12. Doğruluk Kanıtı ve Test	15
13. Nasıl Kurulur ve Çalıştırılır	16
14. Yol Haritası	17
15. İletişim & Hizmet	18

1. Bu Sistemi Şirketinizde Kuruyoruz

Şirketinizin verisi var ama çoğu çalışan SQL bilmediği için ona ulaşamıyor. Hazır bulut çözümleri ise veriyi dışarı (OpenAI vb.) gönderiyor — birçok kurum için kabul edilemez. **Şirket-Sorgu** bu iki sorunu birden çözer: çalışanlar **Türkçe** sorar, cevap **saniyeler içinde** gelir ve **veri makineden hiç çıkmaz**.

Ben, **Harun Gökçe**, bu sistemi **sizin verinize ve ihtiyaçlarınıza göre** kuruyorum.

Sunduğum hizmetler

Kurulum & entegrasyon

Sistemi şirketinizin sunucusuna/bilgisayarına kurar, **kendi veritabanınıza** (PostgreSQL, MySQL vb.) bağlarım. Veri yerinde kalır.

Özelleştirme & eğitim

Şemanıza özel örnekler, terimler ve raporlar; ekibinize kullanım eğitimi.

Bakım & geliştirme

Doğruluğu artırma, yeni soru türleri, performans ve sürüm güncellemeleri.

Özel AI çözümleri

Yalnızca bu sistemi değil; şirketinize özel **ya-pay zekâ ajanları ve otomasyonlar** da geliştiriyorum.

Hangi sektörler için?

Sistem veritabanı-agnostiktir; veri analizi yapan **her sektöre** uyarlanır. Özellikle **gizlilik-hassas** sektörlerde yerel çalışma kritik avantajdır:

Sektör	Örnek kullanım
Perakende & E-ticaret	Satış, ciro, stok, kampanya ve ürün performansı analizi
Muhasebe & Mali Müşavirlik	Müşteri/cari verileri — gizliliği kritik, yerel çalışma şart
Sağlık & Klinik	Hasta/randevu verisi — KVKK gereği veri dışarı çıkamaz
Hukuk Büroları	Dosya/müvekkil verisi üzerinde gizli sorgulama
Finans & Sigorta	Regülasyon altında poliçe/işlem analizi
Lojistik & Kargo	Sevkiyat, teslimat süreleri, bölgesel performans
Üretim & Fabrika	Üretim, bakım, fire ve verimlilik raporları
Turizm & Otelcilik	Rezervasyon, doluluk, gelir (RevPAR) analizi
Eğitim Kurumları	Öğrenci/başarı verisi üzerinde güvenli raporlama
Kamu & Belediye	Vatandaş verisi — yurt içinde, yerelde kalmalı

Şirketinizde kuralım

Kendi verinizle denemek veya kurmak için: **harungokce70@gmail.com** · **0506 155 46 42** · github.com/Harungokc/local-text-to-sql

2. Yönetici Özeti

Bu sistem, bir şirketin kendi veritabanına **doğal Türkçe ile** soru sorabilmesini sağlayan, **tamamen yerel (local) çalışan** bir text-to-SQL yapay zekâ ajanıdır. Kullanıcı “en çok ciro yapan 5 ürün hangisi?” diye sorar; sistem soruyu güvenli bir SQL sorgusuna çevirir, salt-okunur çalıştırır, sonucu işler ve **doğrulanmış** bir Türkçe rapor döndürür.

Tek cümlede değer önermesi

Kurumsal text-to-SQL hâlâ çözülmemiş zor bir problem (Spider 2.0 benchmark’ında GPT-4o bile yalnızca ~%10). Bu sisteme **mühendislik disipliniyle** ve **veriyi hiç dışarı çıkarmadan** yaklaşıldı.

Kimin için?

- **İşletmeler:** SQL bilmeyen ekiplere self-servis veri analizi.
- **Yöneticiler:** anında Türkçe rapor, dışa bağımlılık yok.
- **Gizlilik-hassas kurumlar:** veri makineden hiç çıkmaz (KVKK/GDPR dostu).

Ölçülen sonuç (3B model, dizüstü)

- Temel/orta sorular: **%87–95 doğruluk**
- İleri analitik (pencere fn., percentile): **~%20** (sunucuda 7B/32B ile yükselir)
- Güvenlik testi: **15/15** · Türetilmiş metrik: **3/3**
- Tüm bu sayılar gerçekten ölçüldü — uydurma yok.

Ayırt edici yan tek bir model değil, **mimaridir**: şema linkleme, RAG few-shot örnekleme, çok-katmanlı güvenlik/doğruluk kontrolü, türetilmiş-metrik denetimi ve “sayıyı kod hesaplar, LLM yalnızca yorumlar” ilkesiyle **halüsinasyonsuz** rapor.

3. Sorabileceğiniz Soru Türleri

Sistem şemanızı **çalışma anında** okur; aşağıdaki soru türlerini Türkçe sorabilirsiniz. (İşaretli sayılar bu demoda **gerçekten ölçülmüş** sonuçlardır.)

Satış & ciro

- “en çok ciro yapan 5 ürün hangisi?”
- “şehir bazında toplam satış adedi nedir?”
- “Pinar markasının toplam cirosu ne kadar?”

Oran & pay soruları (ölçülmüş gerçek sonuçlar)

- “İçecek kategorisinin toplam ciro içindeki payı nedir?” → %13,1
- “İstanbul, Antalya’dan kaç kat fazla ciro yaptı?” → ~3,51 kat
- “en çok ciro yapan ürünün toplam içindeki payı nedir?” → %4,1

Neden oran soruları özel?

Çoğu sistem yüzde/pay sorularında **yanlış paydadan** hesap yapar (örn. yalnızca ilk 5 ürünün toplamını alır). Bu sistem ayrı bir doğrulama katmanıyla (K3) **gerçek toplamı** çekip doğru yüzdeyi verir.

Kampanya & indirim takibi

Verinizde kampanya/indirim alanları olduğunda şu sorular yanıtlanır:

- “X kampanyası döneminde ciro ne kadar arttı?”
- “indirimli ürünlerin toplam satışı nedir?”
- “kampanyalı vs kampanyasız ürünlerin cirosu nasıl karşılaştırılıyor?”
- “hangi kampanya en çok ek satış getirdi?”

Dürüst not

Demo veritabanında kampanya verisi **yoktur** — bu yüzden sistem demoda bu soruları (uydurmamak için) güvenle reddeder. Şirketinizin veritabanında kampanya/indirim tabloları olduğunda, sistem şemayı runtime okuduğu için bu sorular **otomatik** yanıtlanır hale gelir.

Karşılaştırma & trend

- “bu ay ile geçen ayın cirosunu karşılaştır”
- “son 7 günde günlük ciro nasıl?”

Stok / müşteri (verinize göre)

- “stok azalan ürünler hangileri?” · “en çok alışveriş yapan müşteriler kim?”

(Bu alanlar veritabanınızda varsa yanıtlanır; yoksa sistem “bu veri yok” der, uydurmaz.)

4. Problem: Text-to-SQL Neden Zor, Gizlilik Neden Önemli?

Zorluk gerçek

Basit akademik testlerde (Spider 1.0) en iyi sistemler ~%86–91 başarıya ulaşır. Ama **gerçek kurumsal şemalarda** (Spider 2.0 — 1000+ kolonlu veritabanları, 100+ satırlık SQL) tablo tersine döner:

Benchmark / Sistem	Doğruluk
Spider 1.0 (basit, akademik)	~%86–91
BIRD (gerçekçi, gürültülü)	en iyi %81.95 / insan %92.96
Spider 2.0 — GPT-4o (gerçek kurumsal)	~%10
Spider 2.0 — o1-preview	~%21

Çıkarım

Model seçimi tek başına belirleyici **değil**. Doğruluğu **mimari** belirler — retrieval, şema linkleme, kontrol katmanları, semantik denetim. Sistemin neden “tek model + tek atış” değil de katmanlı bir pipeline olarak tasarlandığının birinci-elden gerekçesi budur.

Gizlilik: şirketlerin asıl korkusu

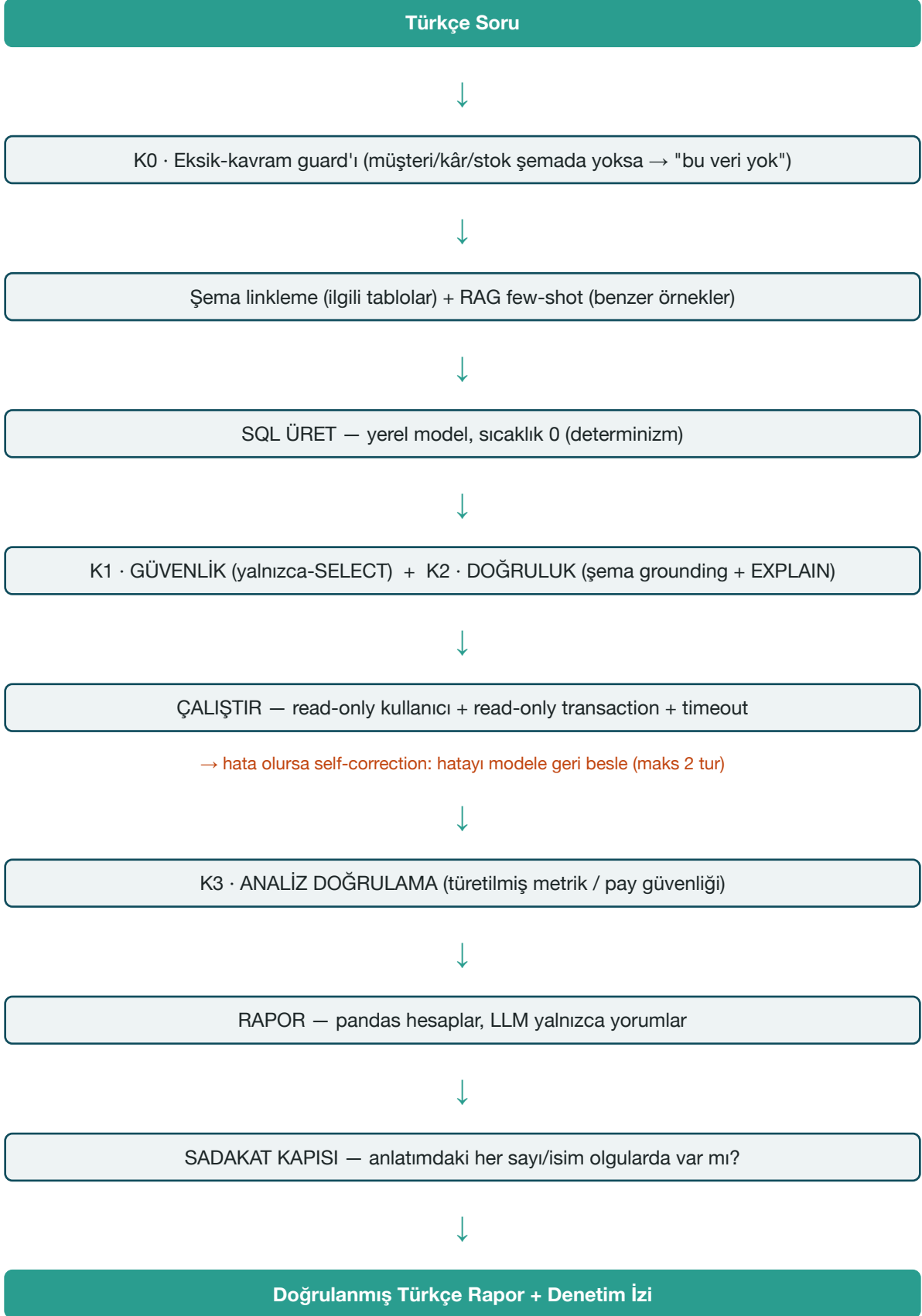
Çoğu hazır çözüm soruyu ve şemayı bir bulut API’sine gönderir — yani **veri şirketten çıkar**. Birçok kurum için bu pazarlıksız bir engeldir. Bu sistem baştan sona **yerel modelle** çalışır: soru, şema ve veri bilgisayardan/sunucudan **hiç dışarı çıkmaz**.

5. Bu Sistemi Ne Ayırıştırıyor?

Sıradan demo	Bu sistem
Bulut API → veri dışarı çıkar	%100 yerel — gizlilik-korumalı
SQL'i körü körüne çalıştırır	Çok-katmanlı güvenlik (read-only kullanıcı + read-only transaction + AST denetimi + timeout)
"Çalışıyor" der, kanıt yok	Dürüst doğruluk ölçümü (kendi gold set + execution accuracy)
Tek model, tek atış	Araştırma-temelli mimari (şema linkleme + few-shot + self-correction)
Sayıyı LLM uydurur	Sayıyı pandas hesaplar , LLM yalnızca yorumlar → halüsinasyon yok

6. Mimari: Uçtan Uca Akış

Sistem bir soruyu, her adımı denetlenen bir hat üzerinden cevaba dönüştürür:



Modüler tasarım

Her katman tek sorumluluk taşır ve ayrı dosyadadır: `sql_guvenlik.py` (güvenlik), `sql_kontrol.py` (doğruluk), `analiz_kontrol.py` (metrik denetimi), `sema.py` (şema), `retrieval.py` (few-shot/RAG), `rapor.py` (rapor), `db_sorgu.py` (read-only DB), `runner.py` (orkestratör).

7. Kontrol Katmanları: Sistemin Kalbi

Tasarımın merkezinde bir ilke var: **her adıma kontrol koymak aşırı mühendisliktir**. Bunun yerine “hata olasılığı $\times$ etki” en yüksek **kritik noktalara** odaklanıldı. Ve sıralama bilinçli: **önce ucuz deterministik kontroller, en son pahalı LLM**.

Kontrol	Ne yakalar	Tür	Maliyet
K0	Şemada karşılığı olmayan kavram (müşteri, kâr, stok) → uydurma yerine “bu veri yok”	deterministik	~0
K1	Yazma/silme/DDL/injection — yalnızca SELECT’e izin	deterministik	~0
K2	Uydurma tablo/kolon (AST grounding) + EXPLAIN ile tip/şema garantisi	deterministik	~0
K3	Türetilmiş metrik hatası (kesik sonuçta yanlış “pay/yüzde”)	deterministik	~1 ek sorgu
Sadakat	Anlatımda olgularda olmayan sayı/isim → reddet, deterministik özete düş	deterministik	~0

Tasarım felsefesi

Kendinden emin yanlış cevap vermektense “emin değilim / bu veri yok” demek daha sağlamdır. Koşul sağlanmazsa sistem metriği bastırır ve çekince ekler. Dürüstlük, hem doğruluk hem güven kazandırır.

8. Halüsinasyonsuz Rapor

Rapor katmanı **hibrit** tasarlandı çünkü “LLM hesaplasın” güvenilirmez (akademik testlerde sayısal doğruluk ~%78):

1. `ozetle()`
DETERMİNİSTİK
pandas hesaplar

2. `icgoru_sec()`
KURAL TABANLI
en önemli bulgular

3. `anlat()`
LLM YALNIZCA DİL
Türkçe yorum

Tüm sayılar (toplam, ortalama, pay, sıralama) **kod** tarafından hesaplanır; LLM yalnızca akıcı Türkçe cümleyi yazar ve “verilen sayı dışında rakam üretme” talimatı alır. Üstüne **sadakat kapısı**: anlatımdaki her sayı/isim yapılandırılmış olgularda yoksa, anlatım reddedilip %100 sadık deterministik özete düşülür.

Gerçek çıktı — "en çok ciro yapan 5 ürün hangisi?"

ÜRETİLEN SQL:

```
SELECT u.ad AS urun, SUM(s.toplam_tutar) AS ciro
FROM satislar s JOIN urunler u ON u.id = s.urun_id
GROUP BY u.ad ORDER BY ciro DESC LIMIT 5
```

SONUÇ: Dana Kuşbaşı 2.687.760 · Beyaz Peynir 2.244.220 · Kıyma 2.140.880 ...

ÖZET : Dana Kuşbaşı 1kg, tüm ciro (66.181.361) içinde %4.1 pay tutuyor.

İZ : K0 ✓ K1 ✓ K2 ✓ Çalıştırma ✓ K3: gerçek toplam ayrı çekildi Sadakat ✓

Not: “pay” hesabı için payda (66.181.361) ayrı, güvenli bir sorguyla çekildi — LIMIT’li sonucun toplamı **değil**. Bu, K3’ün asıl işidir.

9. Veritabanı: Neden PostgreSQL?

PostgreSQL, gizlilik-korumalı ve güvenli bir text-to-SQL hattı için ideal özelliklere sahip:

Özellik	Katkısı
Salt-okunur kullanıcı (GRANT)	Yazma/silme'yi fiziksel olarak imkânsız kılar — en güçlü koruma katmanı
Read-only transaction	Çalıştırma sırasında ikinci savunma hattı
EXPLAIN (yan etkisiz)	Sorguyu çalıştırmadan şema/tip uyumunu %100 doğrular (K2)
<code>information_schema</code>	Şemayı runtime'da okuma → kod içine gömülü DDL yok; kendi DB'nizi bağlayabilirsiniz
Olgunluk & yaygınlık	Kurumlarda zaten standart; ek lisans/maliyet yok

Hangi diğer veritabanlarıyla yapılabilir?

Mimari veritabanı-agnostiktir; SQL diyalekti değişse de yaklaşım aynıdır:

Veritabanı	Not
MySQL / MariaDB	Çok yaygın; read-only kullanıcı + EXPLAIN mevcut, diyalekt farkları küçük
SQLite	Tek-dosya, kurulumuz; küçük/yerel kullanım için ideal
DuckDB	Analitik (OLAP) için çok hızlı; CSV/Parquet üzerinde doğrudan SQL
SQL Server / Oracle	Kurumsal; diyalekt/izin modeli farklı ama aynı kontrol felsefesi geçerli

10. Model & Sunucu: Hangisi Ne Kadar Yeterli?

Aynı kod hem dizüstüde küçük modelle, hem sunucuda büyük modelle çalışır — **tek değişen iki ortam değişkeni** (`YEREL_MODEL` , `VLLM_BASE_URL`).

Model	Boyut	Nerede	Rol
Qwen2.5-Coder 3B	1.9 GB	Dizüstü/PC	Gündelik raporlama (temel/orta) — %87–95
Qwen2.5-Coder 7B	4.7 GB	Sunucu/GPU	Daha yüksek doğruluk, daha çok kullanıcı
Qwen2.5-Coder 32B	~18–24 GB	Sunucu/GPU	İleri analitik (pencere fn., karmaşık analiz)

Sunucuda çalıştırma (7B / 32B)

İleri analitik veya çok kullanıcı gerektiğinde sistem bir sunucuda büyük modelle çalışır. Veri yine **tek-kiracı** ve **dışarı çıkmadan**:

Kiralık GPU

RunPod gibi sağlayıcılarda izole (tek-kiracı) GPU — örn. A6000 ~\$0.49/saat. **İhtiyaç anında aç, işin bitince kapat** → maliyet düşük. 32B'yi rahat taşır.

Yurt içi / dedicated sunucu

KVKK hassasiyeti yüksekse tam dedicated/ fiziksel sunucu (yurt içi). Veri ülke dışına hiç çıkmaz.

Şirketin kendi sunucusu

Donanımınız varsa sistemi **sizin** sunucunuza kurarım — hiçbir dış bağımlılık olmadan.

Servis altyapısı

vLLM ile OpenAI-uyumlu, çok-istekli (batching) servis. Web arayüzü + API hazır.

Bu kurulumu biz yapıyoruz

Donanım seçimi, model kurulumu, güvenli ağ ve sizin verinize entegrasyon — uçtan uca tarafımızdan yapılır. Siz yalnızca soruları sorarsınız.

3B'nin dürüst sınırı

3B; pencere fonksiyonu, grup-başına-top-N, percentile gibi **ileri analitiği** dizüstüde tek başına çözemez (kavramsal sınır). Bu sorular için sunucuda 7B/32B devreye girer. “Küçüklerle başla, gerektiğinde büyüt” stratejisi.

11. Yerel (Local) Çalışmanın Avantajları

Gizlilik & uyum

Veri, şema ve sorular makineden **hiç çıkmaz**. KVKK/GDPR hassas kurumlar için pazarlıksız avantaj.

Sıfır API maliyeti

Token başına ücret yok. Sorgu sayısı arttıkça maliyet **artmaz**. Bulut API'de her sorgu para demektir.

Bağımsızlık

İnternet kesintisi, API kotası, fiyat değişikliği **etkilemez**. Sistem tamamen sizin kontrolünüzde.

Öngörülebilirlik

Sabit donanımda **tutarlı** gecikme; dış servis yavaşlaması yok.

12. Doğruluk Kanıtı ve Test

Sistem “çalışıyor” demekle yetinmez; **ölçer**. Dört bağımsız test eksenini:

Test	Ne ölçer	Sonuç
test_guvenlik.py	Yazma/injection reddi (K1)	15/15
test_kontrol.py	Uydurma kolon/tablo reddi (K2)	10/10
gold_set.py	Uçtan uca execution accuracy	7/8 (~%87)
metrik_test.py	Türetilmiş metrik doğruluğu (K3)	3/3

Kontrol katmanlarının kanıtı

K3 olmasaydı sistem “Dana Kuşbaşı, toplam ciro içinde %25.0” derdi (yanlış — sadece 5 satırın toplamı). K3 sayesinde **doğru** cevap: “%4.1” (gerçek toplam 66.181.361 ayrı çekilerek). Bu fark, mühendisliğin doğruluğa katkısının somut kanıtıdır.

13. Nasıl Kurulur ve Çalıştırılır

Gereksinimler

- Python 3.11+ • PostgreSQL • [Ollama](#) (yerel model çalıştırıcı)

Adım adım

```
# 1) Repoyu klonla
git clone https://github.com/Harungokc/local-text-to-sql.git && cd local-text-to-sql

# 2) Sanal ortam + bağımlılıklar
python3 -m venv .venv && .venv/bin/pip install -r requirements.txt

# 3) Yerel modeli indir (Ollama açık olmalı)
ollama pull qwen2.5-coder:3b

# 4) Demo veritabanını kur
createdb sirket_demo
export DATABASE_URL="postgresql://$USER@localhost:5432/sirket_demo"
.venv/bin/python demo_veri.py

# 5) Few-shot örneklerini yükle
export SORGU_DATABASE_URL="postgresql://$USER@localhost:5432/sirket_demo"
.venv/bin/python seed_sema.py

# 6) Web arayüzünü başlat → tarayıcıda http://127.0.0.1:9000
.venv/bin/python -m uvicorn api:app --port 9000
```

Üretimde dikkat

Uygulama **salt-okunur** bir DB kullanıcısı kullanmalı. Gerçek şirket verisiyle kurulum, doğru izinler ve güvenli ağ ayarlarını içerir — bu kurulumu sizin için biz yapıyoruz.

14. Yol Haritası

Tamamlanan

- Uçtan uca pipeline + 5 kontrol katmanı
- Web arayüzü + REST API + terminal
- Dürüst doğruluk ölçümü (4 test eksenli)
- Açık kaynak repo + dokümanlar

Sıradaki (Faz 3–4)

- Koşullu LLM-critic + güven skoru (“emin değilim”)
- Intent-based retrieval + kolon budama (büyük şema)
- Semantik metrik katmanı (sabit iş tanımları)
- Sunucuda 32B + grafik üretimi

15. İletişim & Hizmet

Bu sistem, kurumsal text-to-SQL'in zor bir problem olduğunu kabul ederek ona **mühendislikle** ve **veriyi hiç dışarı çıkarmadan** yaklaşır. Doğruluğu modelden değil; katmanlı kontrol, RAG few-shot ve halüsi-nasyonsuz rapor **mimarisinden** alır.

Sonuç: dizüstünde bile çalışan, gündelik iş sorularını %87–95 doğrulukla yanıtlayan, sınırlarını dürüstçe belgeleyen bir asistan — ve gerektiğinde sunucuda büyük modele ölçeklenen bir mimari.

İletişim — Şirketinizde Kuralım

Ad: Harun Gökçe — Freelance Yapay Zekâ & Yazılım Geliştirici
E-posta: harungokce70@gmail.com
Telefon: 0506 155 46 42
GitHub: github.com/Harungokc/local-text-to-sql

Bu sistemi şirketinizin verisiyle denemek veya kurmak için iletişime geçin.

Açık kaynak: github.com/Harungokc/local-text-to-sql · Bu sistemi şirketinize biz kuralım.